KonaOS

RETRO SOUL, MODERN FLOW

KonaOS Programmer's Guide

Writing, building, running and debugging native EXE programs for KonaOS with the SDK, without modifying the operating system.

Edition 1 October 2026 Draft

FRONT MATTER

Audience and authority

Who this guide is for

Developers who write programs that run *on* KonaOS: tools, games, players and utilities built with the SDK. You do not need to know how the nucleus, the drivers or the boot path work, and this guide does not tell you. If you are changing the operating system itself, the architecture documents are your starting point instead.

What you need to know already

C as SDCC compiles it for the Z80, and enough Z80 to read an address map. Nothing about CP/M is assumed; KonaOS has no drive letters, FCBs or USER areas.

Source of truth

The SDK headers and `docs/sdk.md`, `docs/abi.md` and `docs/exe-format.md` are authoritative, with `tui.md`, `gfx-sdk.md`, `snd-sdk.md` and `kres-sdk-design.md` for the optional libraries. This guide reorganises what they say by what a program author needs, in the order they need it. Where the two disagree, the headers win and this guide should be corrected.

CONTENTS

What is in this guide

PART I · FOUNDATIONS

1 What you are writing
2 Getting started
3 The program contract
4 The memory model

PART II · BUILDING AND THE SDK

5 Building
6 The SDK surface
7 Far code and far data
8 I/O performance habits
9 Device events
10 Streams, measurement and planning
11 Text user interfaces
12 Graphics
13 Music and sound
14 Readers and resource containers

PART III · RUNNING

15 Running and debugging
16 Limits you will hit
17 Examples walk-through

APPENDICES

A Quick reference of the SDK headers
B Error codes
C EXE geometry rules that affect link success

FRONT MATTER

Reading paths

Everyone should read chapter 2. After that, read what your program needs.

YOU ARE WRITING	READ
a small utility: console and arguments	2, 3, §6.1; skim 4
a program that uses files	the above, then §6.2, 8 and 10
a program too large for its bank	4 in full, then §7.8; the rest of 7 and Appendix C to lay out far memory by hand
a program that reacts to time or devices	§6.5 and 9
a player or other sustained reader	8, 9 and 10
a full-screen text interface: panels, menus, forms	§5.1, then 11 and §1.6
a program that draws: pictures, shapes, bitmap text	§5.1, then 12
a program that plays notes, songs or sound effects	§9.1, then 13 and §14.1; §9.7 for raw register writes
a program with many data files, or a game with assets	14, and the open-file limits in 16
anything, and you hit an error	16, then Appendix B for the code or Appendix C if the packer refused

PART I

Foundations

What a KonaOS program is, how to get one running, what the system promises it at entry and exit, and the memory it lives in.

Read chapters 3 and 4 before writing anything larger than the template. Most link failures and most mysterious crashes trace back to one of the rules in them.

PART I · 1

What you are writing

A KonaOS program is a **native EXE**: a `KEX2` container holding one primary image and, optionally, up to eight auxiliary segments. The system loads the primary image into one free **primary bank**, one of banks 0-5, and enters it. You never choose the bank, and the same bytes run unchanged in any of them: every image is linked at `0100h` and there is no relocation.

KonaOS is **single-tasking**. A program runs until it exits; if another program started it, that parent is suspended and resumes when the child finishes. A program may start a child of its own with `run()`, up to four user processes deep. There is no background execution, no `wait` and no asynchronous spawn.

There is **no hosted C library**. The SDK is the whole runtime: `crt0`, `libkos` (the system calls and protocol clients) and `klibc` (strings, formatting and a small buffered file layer). There is no `malloc`, no floating point and no POSIX emulation. Every library function is its own archive member, so a program links only what it calls.

PART I · 2

Getting started. Your first hour with KonaOS

This chapter is the first hour in one place: what you can and cannot do, a program, how to build it, how to run it, and what to do when either step fails.

2.1 What you get and what you do not

YOU GET	YOU DO NOT GET
56 KiB of private memory in your own bank	a heap allocator
far code and far data in other banks, through the SDK	bank numbers, latch values or direct bank switching
files, devices and the namespace through handles	drive letters, FCBs or direct hardware ownership (except by exclusive bind)
`printf`, `snprintf`, string functions, buffered `FILE` streams	floating point, an integer multiply helper (§5.4) or initialised writable statics
a monotonic clock and device events	threads, signals or a scheduler
optional libraries for text screens, V9958 graphics, music and resource containers	a GUI toolkit, windows on the bitmap screen, or a mouse

THE FIRST GOTCHA

There are **no initialised writable statics**. `static int level = 5;` or a file-scope `int level = 5;` is refused when the program is packed. Every static starts at zero: declare tables `const`, and set writable variables at run time.

2.2 The minimal program

```
#include "kos.h"

int main(int argc, char **argv)
{
        printf("Hello, KonaOS!\r\n");
        return 0;
}
```

No entry assembly, no manifest and no kernel header. The return value of `main` becomes the exit status.

2.3 Build it

IN ONE SENTENCE

Copy `template/` from the SDK package, change the name in the `exe_program` line, put your `.c` next to the Makefile, and run `make`.

The result is `build/user/<name>.exe`. The SDK package itself is built in the KonaOS tree with `make sdk` and unpacks to `kos-sdk-<version>/`; its `VERSION` file reads `<version> abi <n>`, and a program built against another ABI version is refused by the loader. A Makefile outside `template/` works too if it includes the SDK's `mk/kos.mk` and sets `KOS_SDK_ROOT`.

Inside the KonaOS tree, the same line goes in a Makefile that includes `user/sdk/exe.mk`:

```
$(eval $(call exe_program,hello,user/sdk/examples,hello.c,STACK=256,STAGE=1))
```

`STAGE=1` is only for programs that go into a ROM image's `/system`.

2.4 Run it

Copy the `.exe` to the SD card under an upper-case 8.3 name, for example `/BIN/HELLO.EXE`. Then:

- **From the shell**, type `hello`. The shell looks in `.`, `/BIN` and `/system`, with or

without `.exe`, so a program in `/BIN`, or in the directory you have `cd`'d to, runs by name.

- **From the monitor**, give the full path: `X /fs/BIN/HELLO.EXE`.

```
/> hello one two
Hello, KonaOS!
```

Both report the program's exit status after it exits.

SHELL PATHS VERSUS REAL PATHS

The shell shows `/fs` as `/`, so at the shell prompt `/BIN/HELLO.EXE` means `/fs/BIN/HELLO.EXE`, and `//` names the real root. That is display sugar only. **In code, always use the real path**: `open("/fs/BIN/DATA.TXT", KFS_O_READ)`.

2.5 If the packer refuses

The packer applies every check the loader would, so a bad program fails at build time with the rule it broke. Appendix C lists them all. The three common causes are:

- initialised writable data (§2.1);
- the stack reservation colliding with BSS, often after raising `STACK`;
- image, BSS and stack together over the 56 KiB primary bank.

2.6 If it runs but a call fails

Every SDK call returns a negated `KOS_E*` code on failure. Print it and exit non-zero:

```
int h = open("/fs/DATA/LEVELS.DAT", KFS_O_READ);

if (h < 0) {
        printf("open failed %d\r\n", -h);
        console_flush();
        return 1;
}
```

Look the number up in Appendix B; chapter 16 lists the usual causes of each. If the program stops without printing anything, use the marker pattern in §1.4.

PART I · 3

The program contract

3.1 Entry

Write `int main(int argc, char **argv)`, or `int main(void)` if the program ignores its arguments. The system enters `crt0` at `kos_entry` with `HL = 0080h`, the address of the startup block. `crt0` then:

1. stores the block's address for `startup()`;
2. checks `version == 1` and `tail_length ⩽ 125`, treating anything else as an empty command line;
3. copies the tail into the SDK's own 127-byte buffer and splits it there, in place, so `argv` points into your data and survives later system calls;
4. calls `main(argc, argv)` with `argc` clamped to `KOS_ARGV_MAX` (16);
5. calls `exit` with the return value.

3.2 The startup block

```
0080h  u8   version = 1
0081h  u8   tail_length   0..125
0082h  u8   tail[126]     tail_length raw bytes, then one NUL
```

The tail is the raw text after the program name, byte-exact. The kernel never splits or interprets it. `startup()` returns a pointer to this block if you need the raw form.

3.3 Argument splitting

ARGV[0]

`argv[0]` is the first argument, not the program name. The kernel passes no name, so a program run with no arguments sees `main(0, argv)`.

The shell and every program share one splitter, `splitargs()`, so a child's `argv` always matches the words the shell ran it with:

- words are separated by spaces or tabs; leading and trailing separators are ignored;
- a single quote toggles a literal section: everything inside is ordinary text;
- a double quote toggles a quoted section where separators are literal and `\"` and `\\` are escapes;
- outside quotes, `\x` is a literal `x`, so `a\ b` is one word;
- adjacent pieces join: `ab"c d"e` is the single word `abc de`;
- `''` and `""` are empty arguments; an unterminated quote runs to the end.

These rules are runtime policy, not ABI. They may change in a later SDK without any kernel change.

3.4 Exit

Return 0 for success and non-zero for failure. Returning from `main` and calling `exit(status)` are equivalent; the shell and the monitor report the status, and a parent receives it from `run()`.

Exit flushes the console buffer and, if the program uses streams, every open stream; then the system releases the process slot, the bank, every handle, every extended-memory object and every segment, and resumes whoever started you.

STARTED BY	WHAT HAPPENS TO YOUR STATUS
another program's `run()`	returned as `run()`'s result
the shell	reported by the shell after the child exits
the monitor's `X`	printed as `exec: exit status 0xssss`
the boot launcher	printed as `boot: <path> exit <n>`; then the image's policy applies (§1.2)

A program that faults loses console output it has not flushed. A raw `RET` from the entry point, which `crt0` never does, ends the process with `KOS_EFAULT`.

3.5 What a child inherits

A program started with `run()` gets a fresh bank, its own page zero, standard handles 1 and 2, and a copy of its parent's current directory. Nothing else passes: no other handles, no extended-memory objects. A child's `chdir` never affects the parent.

While the child runs, the parent's handles stay allocated. The 12-entry handle table is shared, so a parent holding many handles leaves fewer for its child.

PART I · 4

The memory model

KonaOS gives each running program one primary bank and a small set of shared or optional regions. You never choose the bank number; the system does.

```
Your process (mode 10, primary bank Q)

0000h-00FFh     page zero, owned by the system
                (RST 30h vector at 0030h, startup block at 0080h)
0100h-015Ch     the SDK's fixed low data: syscall scratch and
                argument blocks (93 bytes)
015Dh           your code begins (crt0 first)
...             code and const data, then BSS: zeroed statics,
                the argv copy, buffers you declare
...             free memory, not cleared
E000h - STACK   the stack reservation; the stack grows down
                toward it from E000h
E000h-FFFFh     common memory (always visible, not yours)
```

4.1 The primary bank, `0000h-DFFFh`

This is your private 56 KiB. Code, constant tables, static data, buffers you manage yourself and the stack all live here. The system loads your image at `0100h` and reserves `STACK` bytes below `E000h` for the stack, the value you declared at build time (default 256).

Two properties of this memory catch new authors:

- **Every static starts at zero.** Initialised writable data is not supported: a file-scope `int level = 5;` is refused when the program is packed. Declare constant tables `const`, which places them with the code, and set writable variables at run time.
- **Nothing else is cleared.** The loader zeroes your BSS and nothing more. Memory between the end of your BSS and the stack holds whatever the

previous program left there. Initialise any buffer you carve out of it.

4.2 Common memory, `E000h-FFFFh`

Always mapped. The system keeps its gates, interrupt state and shared services there. Do not write to it and do not depend on its contents or layout, which change between builds; treat it as present but not yours. The only reason to name it is that an IPC buffer may legally lie there.

4.3 The far aperture, `8000h-BFFFh`

When you use far code or mappable far data, the SDK temporarily maps another bank into this 16 KiB window. It does so with a single latch write, not a system call. While the aperture is open:

- the window shows the far segment or object, not your primary bank;
- your stack and any data you still need must live outside `8000h-BFFFh`;
- only one far thing is visible at a time: the **one-window rule**.

The aperture stays open across system calls: the gate restores exactly the mapping it found.

Far-code segments and `DATAn` segments always appear at `8000h`. A mappable object from `xalloc` appears at `8000h` or `A000h`; `xmap` tells you which. If your program never uses far facilities, the aperture never opens and `8000h-BFFFh` is ordinary primary-bank memory.

4.4 What this implies for you

RULE	WHY IT EXISTS
With hand-declared far segments, keep the stack reservation at or above `C000h`, so `STACK` is at most 8 KiB	So the aperture cannot sit on your stack. The packer checks it.
With hand-declared far segments, keep primary image and BSS below `8000h`	So the aperture does not overlay your own code and data. The packer checks it.
Syscall argument blocks lie in `0100h-1FFFh`	That range stays visible while the system runs. The SDK keeps its blocks there for you; this matters only if you issue `RST 30h` yourself.
Data buffers for `read`, `write` and IPC lie in `0100h-DFFFh` or common memory; one IPC buffer is at most 4,096 bytes	The system copies from the bank it saved at the call.
Do not keep live data in `8000h-BFFFh` across a banked call or `farmap`	That window can change underneath you.
A buffer passed from far code while the aperture is open must lie wholly inside or wholly outside it	A buffer that crosses the aperture's edge is refused.
Size `STACK` for your deepest call chain plus any event handler	Event handlers run on your stack, below a 24-byte delivery frame.
In a placed program, name every event handler's source in `RESIDENT=`	A handler runs wherever the program was interrupted, far code included (§7.8).
Copy the startup block's tail before passing it to a system call	It lies below `0100h`. `crt0` already does this for `argv`.

4.5 Far memory in one paragraph

A program that does not fit its 56 KiB bank can be **placed** automatically: the toolchain divides its modules between a resident zone that is always visible, a home zone at `8000h-BFFFh` of its own bank and far segments, and routes calls between zones through generated veneers, so the sources stay plain C. A program can instead declare extra code by hand, as far-code segments reached with ordinary C calls to `__banked` functions (or `farcall`). Extra data is either a mappable segment (`farmap`, then ordinary pointers, then `farunmap`) or an opaque object you copy with `xread`/`xwrite`. You never see bank numbers or latch values; the SDK owns the mapping. Each far-code segment and each mappable data segment costs one auxiliary bank for the life of the program, and at most five fit. Chapter 7 has the detail.

4.6 What you can ignore

Bank 7 (the nucleus), bank 6 (drivers), ROM pages, the fixed-core and cold-half split, and the residency ledgers. Those are the system's problem. Your contract is the layout above plus the SDK.

PART II

Building and the SDK

The build rule and its options, the functions the SDK gives you grouped by what you need them for, far memory, the habits that keep I/O fast on a 10 MHz machine, the two facilities specific to KonaOS, device events and planned streams, and the optional libraries for text screens, graphics, music and resource containers.

PART II · 5

Building

```
include user/sdk/exe.mk      # in the tree; the standalone SDK includes mk/kos.mk
$(eval $(call exe_program,NAME,DIR,SOURCES[,OPTION=VALUE ...]))
```

`SOURCES` are relative to `DIR`: `.c` goes through SDCC, `.asm` through `sdasz80`. The rule links `crt0` and the runtime object first, then your objects, then `libkos.lib`; derives the manifest from the link map; and packs and validates `build/user/NAME.exe`. You never write a manifest or enter a size by hand. When the packer refuses a program, Appendix C names the rule.

5.1 Options that matter to you

OPTION	DEFAULT	USE IT WHEN
`STACK=N`	256	your call depth or local arrays need more; with events, add 24 bytes for the delivery frame plus the handler's own depth; at most 8 KiB in a far program
`RESIDENT=a.c:b.c`	none	the program is placed (§7.8) and has event handlers: name their sources, and any other code that must run while a far segment is mapped
`PLACE=mod=zone:…`	none	you want to pin a source or a library member to `resident`, `home` or `farN`; most placed programs need none
`FAR1=a.c:b.c` … `FAR4=`	none	you lay out far code by hand instead of letting the toolchain place it; each is one far-code segment linked at `8000h`, at most 16 KiB, and the program is then not placed
`DATA1=` … `DATA4=`	none	you have large `const` tables or zeroed arrays to reach by pointer; each is one mappable segment, at most 16 KiB
`BLOBS=a.bin:b.bin`	none	you want binary payloads in the EXE, reached by copy; at most 32 KiB each
`STAGE=1`	off	the program goes into a ROM image's `/system`
`INCLUDES=a:b`	empty	your headers live outside the program directory
`LIBS=a:b`	empty	you use optional SDK libraries, in link order: `tui` or `tui_widgets:tui` (chapter 11), `gfx` (chapter 12), `snd` (chapter 13), `kres` (chapter 14)
`TUI_GRID=CxR`	`85x26`	your TUI program should use a larger console; `128x35` is the only other size, and any other is a build error

`FARn` and `DATAn` are numbered from 1 without gaps. The far-code segments come first in the container: segment *n* is `FARn`, and that number is what your `__banked` definitions name. `ENTRY`, `NO_CRT0`, `LOWBASE`, `CODEBASE`, `SCRATCH` and `SEGMENTS` exist for system tests and special cases; ordinary programs leave them alone.

5.2 Include rules

A program compiles with exactly:

```
sdcc -mz80 -c --nostdinc --Werror -I abi -I user/sdk/include -I <program dir>
```

- `#include "kos.h"` brings in `kos/console.h`, `fs.h`, `ipc.h`, `process.h`, `seg.h`, `xmem.h` and `klibc/string.h`, `fmt.h`, `stdio.h`.

- Include `kos/timer.h`, `kos/event.h`, `kos/stream.h`, `kos/sound.h` or `kos/diag.h` yourself when you use them, `tui.h` or `tui_widgets.h` for the text-screen library, and `kos/gfx.h`, `kos/snd.h`, `kos/reader.h` or `kos/kres.h` for the other optional libraries.
- Your own headers resolve from the program directory and `INCLUDES=`. Nothing under `kernel/`, `drivers/`, `formats/` or `platform/` is reachable, by design.
- `--Werror` is on: every warning stops the build.

5.3 What links what

`libkos` is an archive: a member links only when a symbol it defines is needed. When a symbol is missing, find the call in this table.

A PROGRAM CALLS	MEMBERS LINKED
anything	`kos_crt0`, `kos_runtime`, `args`
`putchar`, `console_write`, `_read`, `_poll`, `_flush`	`console_proto` and the IPC gates
`console_render`, `console_info`	their own members and the IPC gates
`getchar`, `console_puts`, `console_readline`	`console_io`, plus the above
`open`, `read`, …	`fs` and the IPC gates
`truncate`	`truncate` and the IPC gates
`run`	`exec` and the IPC gates
`timer_*`	`timer` and the IPC gates
`xalloc` and friends	`xmem`, `kos_xmemgate`
`seginfo`, `segmap`, `farcall`, `farmap`, `blob` / `xload`	`seg` / `xload`
a `__banked` call	`kos_bcall`
`printf`	`printf`, `strlen` and the four integer formatters
`/` or `%` on integers	the division helpers
a product of two variables, or any 32-bit product	nothing: the link fails (§5.4)
`fopen`, `fclose` and the stream calls	`stdio` group (about 2.2 KiB of BSS for the pool)
`sound_*`	the sound client and the IPC gates
`tui_*`, with `LIBS=tui`	the `libtui` members you call, and its cell map (§11.2)
`gfx_*`, with `LIBS=gfx`	the `libgfx` members you call; no V9958 port code unless you draw (§12.2)
`snd_*`, with `LIBS=snd`	the `libsnd` members you call; a notes-only program links no ZTR or ZVG decoder (§13.1)
`kos_reader_from_*`	that one adapter and what it reads through (§14.1)
`kres_*`, with `LIBS=kres`	the read path alone for a sealed reader; mutation, building and compaction only when called (§14.4)

5.4 There is no multiply

LINK FAILURE

The runtime provides division helpers but **no multiply**. SDCC turns a product of two variables, or any 32-bit product, into a call to `__mulint` or `__mullong`, which no library member defines, so the program does not link.

A product of a variable and a constant links only when the constant is a power of two, which compiles to a shift, or when only the low byte of the result is used. For anything else, write the shift-and-add loop, as the SDK's own `stream_plan` does:

```
static unsigned long mul16(unsigned int a, unsigned int b)
{
        unsigned long r = 0, x = a;

        while (b) {
                if (b & 1)
                        r += x;
                x <<= 1;
                b >>= 1;
        }
        return r;
}
```

PART II · 6

The SDK surface

Every wrapper returns its result, or a negated `KOS_E*` code on failure. Check for `< 0`.

6.1 Console and process

```
void exit(int status);
int  getinfo(struct kos_info_record *record);   /* pid, bank, flags */
int  putchar(int ch);
int  printf(const char *fmt, ...);              /* %s %c %d %u %x %ld %lu %lx %% */
int  console_puts(const char *text);            /* no newline added */
int  console_readline(char *buffer, unsigned int capacity);
int  getchar(void);                             /* waits */
int  console_poll(void);
int  console_flush(void);
int  run(const char *path, const char *args);   /* child's status or -errno */
void idle(void);                                /* halt until an interrupt */
void idle_input(void);                          /* paced wait for input */
void idle_input_ready(void);                    /* input arrived */
```

Output is line-buffered in a 128-byte buffer. It is written when it fills, at `'\n'`, before every console read, before `run`, and at exit. So a prompt without a newline still appears before the input it asks for, and a child never prints ahead of its parent. `console_readline` edits one line of up to 80 characters with backspace and DEL. A process without standard handles gets `-KOS_EINVAL` from every console call.

The console uses code page 850: bytes `20h-FFh` are printable.

Wait without spinning. A program waiting for something should call `idle()` between empty polls. It halts the CPU until the next interrupt, at least once per timer tick (156.25 Hz), instead of looping through system calls. A spinning loop's bus activity falls in the audio band and the sound card picks it up. Events are still delivered while idling, so music driven by timer events keeps playing. Where the system has no timer or interrupts are off, `idle()` returns at once.

Wait for keys more slowly still. Even one poll per tick is audible. For keyboard input, call `idle_input()` between empty polls and `idle_input_ready()` when input arrives. While nothing is typed it waits 16 interrupts between polls, about ten polls a second; for about a second after a key it waits one interrupt, so typing responds per tick. A first key after a pause is read within about 100 ms, and no key is lost: keys wait in the IO controller's queue. `getchar`, `console_readline` and the shell's prompt already wait this way.

Low-level console calls. `console_info` reports the console's text grid and the attribute bits it supports; `console_render` submits one render transaction in the layout of `kos/console_render.h`. The text-screen library (chapter 11) is built on both, and a program that uses it never calls them. A console without the render extension, such as the serial console, answers `-KOS_ENOTSUP`.

6.2 Files

```
/* flags: KFS_O_READ WRITE CREATE EXCL TRUNC */
int  open(const char *path, unsigned char flags);
int  read(unsigned int h, void *buf, unsigned int len); /* 0 = EOF */
int  write(unsigned int h, const void *buf, unsigned int len);
long seek(unsigned int h, long offset, unsigned char whence);
int  sync(unsigned int h);
int  truncate(unsigned int h, unsigned long length);
int  close(unsigned int h);                        /* sync, then release */
int  stat(const char *path, struct kfs_stat *out);
int  mkdir(const char *path);
int  rmdir(const char *path);
int  unlink(const char *path);
int  rename(const char *from, const char *to);
```

Files live under `/fs` (the SD card, writable) and `/system` (the ROM store, read-only: writes answer `KOS_EROFS`). I/O is byte-oriented; short reads are valid; positions are 32-bit. Card names are 8.3. `rename` across the two roots is `KOS_EXDEV`: the system never turns it into a copy. Truncation is `KFS_O_TRUNC` at open, or `truncate` to a given length on an open handle. The machine has no clock, so file times are 0.

6.3 Namespace and devices

```
int bind(const char *path, unsigned int flags, unsigned char *type);
int call(unsigned char handle, struct kos_ipc_request *req);
int unbind(unsigned char handle);
int list(const char *path, unsigned int *cookie,
         struct kos_ns_entry *out, unsigned int max_entries);
int chdir(const char *path);
int getcwd(char *buffer, unsigned int length);
```

One tree with three roots: `/fs`, `/system` and `/dev`, matched case-insensitively. `bind` resolves a name to a handle and reports its protocol type; `call` performs one protocol operation with a 32-byte frame and at most one buffer. `list` enumerates a directory or container in 28-byte entries with a 24-byte name. Devices you may meet:

PATH	WHAT IT IS
`/dev/console`	the console; already bound as handles 1 and 2
`/dev/timer`	the monotonic clock (§6.5)
`/dev/serial/0`	the serial port
`/dev/input/keyboard`	keyboard text
`/dev/video`	the V9958 display; held exclusively, it can deliver one event per frame (chapter 9). `libgfx` holds it this way (chapter 12)
`/dev/sound`	the sound card: four PSGs and a DAC (§9.7, chapter 13)

`call` has two result channels. A request that never reached the IPC layer is mapped to a negative return; a driver's own error comes back as a negative status. Both are negated `KOS_E*` codes, and a driver error is never confused with a malformed request.

To drive hardware beyond its protocol, bind it with `KOS_BIND_EXCLUSIVE`. The driver stands aside until you unbind, then restores the device. A device that does not support takeover answers `-KOS_ENOTSUP`; one already held exclusively answers `-KOS_EBUSY`.

6.4 Extended memory and segments

Only if you need more than the primary bank or code in another bank. The calls are in `kos/xmem.h` and `kos/seg.h`; chapter 7 explains when to use which.

```
int xalloc(unsigned int size, unsigned int flags);  /* KOS_XMEM_FLAG_MAPPABLE */
int xread(unsigned int h, unsigned int off, void *buf, unsigned int len);
int xwrite(unsigned int h, unsigned int off, const void *buf, unsigned int len);
int xfree(unsigned int h);
void *farmap(unsigned int n, unsigned char *prev);
void farunmap(unsigned char prev);
int  blob(unsigned int n);
int  xload(const char *path, unsigned int flags);
```

Extended memory is not pre-zeroed. Sizes round up to whole 8 KiB slots.

6.5 Timer and events

```
#include "kos/timer.h"
int  timer_open(void);
int  timer_now(int handle, uint32_t *now);     /* 25.6 µs counts */
uint32_t timer_us(uint32_t counts);
int  timer_wait_us(int handle, uint32_t us);

#include "kos/event.h"
int notify(int handle, kos_event_handler_t handler);
int timer_event(int handle, uint32_t period_counts);   /* 782 = 20 ms */
```

The clock wraps after about 30.5 hours; take elapsed times as unsigned differences. `timer_us` is valid for spans under about 71 minutes. `timer_wait_us` polls, and returns `-KOS_ETIMEDOUT` rather than hanging if the clock stops. A build without the timer driver answers `-KOS_ENOENT` or `-KOS_ENODEV` from `timer_open`.

Events let a device call into your program when something happens, without a polling loop. Chapter 9 covers them in full.

6.6 Buffered streams

```
FILE *fopen(const char *path, const char *mode);   /* "r" "w" "a", 'b' accepted */
size_t fread(void *p, size_t size, size_t n, FILE *f);
size_t fwrite(const void *p, size_t size, size_t n, FILE *f);
int fgetc(FILE *f);   char *fgets(char *dst, int size, FILE *f);
int fputc(int c, FILE *f);   int fputs(const char *s, FILE *f);
int fseek(FILE *f, long off, int whence);   long ftell(FILE *f);
int fflush(FILE *f);   /* NULL flushes the console and every stream */
int fclose(FILE *f);
```

An ISO C subset with no allocation: a fixed pool of four streams, each with a 512-byte buffer. A transfer of a buffer or more bypasses the buffer. Pending writes leave at `fclose`, `fflush`, a full buffer and at exit.

`kos/stream.h` adds refill timing and rate planning for programs that must read at a sustained rate. Chapter 10 covers the stream layer in full.

PART II · 7

Far code and far data

If your program fits in the 56 KiB primary bank, keep it all there. When it does not, there are two ways on. The simpler is to let the toolchain place it (§7.8), which changes nothing in the sources. The other is to lay out far code and data by hand, as §7.1-7.7 describe. Hand-declared far memory costs primary space: the image and BSS must then stay below `8000h`, which leaves about 32 KiB.

7.1 A worked example: `fardemo`

The smallest far-code program, in the SDK's `examples/`. `fardemo.c` is the primary half; `fardemo_far.c` is linked as segment 1:

```
$(eval $(call exe_program,fardemo,user/sdk/examples,fardemo.c,FAR1=fardemo_far.c))
```

```
/* fardemo.c */
extern int far_main(int a, int b) __banked;   /* in fardemo_far.c */

result = far_main(a, b);                      /* a plain C call */
```

The far half prints from a string that lives inside the aperture, calls back into primary code and returns a result. Run, it prints:

```
fardemo: begin
fardemo: token 50
fardemo: hello from the far bank
fardemo: far result 502B
fardemo: PASS
```

The token depends on which bank the loader chose; your program should never interpret it. The rest of this chapter explains each part.

7.2 Which kind of far memory

KIND	DECLARED	REACHED BY	COSTS
`FARn=`	build time, C	plain calls to `__banked` functions	one auxiliary bank
`DATAn=`	build time, C	`farmap(n)`: plain pointers until `farunmap`	one auxiliary bank
`BLOBS=`	build time, binary files	`blob(n)` gives a handle; `xread`/`xwrite`	slots only; a copy per access
`xalloc`	run time	`xread`/`xwrite`, or `xmap` + aperture if mappable	slots, or an aperture slice if mappable
`xload(path)`	run time, a file	its handle, as for `xalloc`	one copy of the file; at most 32 KiB, 16 KiB mappable

7.3 Banked calls

```
extern int far_main(int a, int b) __banked;   /* defined in a FAR1= source */
result = far_main(a, b);
```

The SDK's trampoline maps the segment, calls, and restores your mapping. Only the first call into a segment costs a system call; later calls reuse its token. Far code may call primary code directly, make banked calls of its own and make system calls. A banked call that names no far segment returns `KOS_RT_EFARMAP` (`30h`) without running.

7.4 Far data with `farmap`

```
unsigned char saved;
const unsigned char *table = farmap(1, &saved);   /* the DATA1 segment */

if (table == (const unsigned char *)0) {
        return 1;        /* saved holds the negated error */
}
level = table[level_index];   /* a plain access */
farunmap(saved);              /* restore the caller's mapping */
```

A `DATAn` source may hold `const` objects and zeroed objects only. An initialised writable object is refused at build time, by name.

7.5 The one-window rule

There is one aperture, and far code shares it with mapped far data: a `__banked` function and a mapped segment are never visible together. So:

- primary code maps data and uses pointers;
- a `__banked` function reaches far data only by copy, through `xread` or values the primary code copied out first;
- `farunmap` before a banked call, `farmap` again after. Each mapper restores exactly the mapping it saved.

7.6 Bank budget

Each far-code segment and each mappable data segment takes one auxiliary bank; your own bank never counts. With six application banks, at most five such segments fit, and fewer when a parent process holds banks. A program that asks for more is refused when it is run (`run` returns `-KOS_ENOSPC`) and nothing is allocated.

7.7 Cost numbers at 10 MHz

OPERATION	COST
a banked call	about 31 µs
access through an open window	nothing extra
`xread`, fast slots	about 21 T-states per byte
`xread`, slow slots	about 287 T-states per byte

Walk a large table by mapping it. Read a few bytes from many places by copying. Keep banked calls out of inner loops.

7.8 Automatic placement

A program that does not fit its bank, and declares no `FARn=` or `DATAn=` segments, is placed by the toolchain. Every object module, one of your sources or a linked library member, gets a zone:

ZONE	ADDRESSES	HOLDS
resident	`0100h-7FFFh`, and `C000h-DFFFh` below the stack	everything that may run or be touched while a far segment is mapped: `crt0` and `libkos`, all data and constants, your event handlers and what they call
home	`8000h-BFFFh` of your own bank	code used only while the aperture is closed
far	segments of up to 16 KiB, each mapped at `8000h-BFFFh` in turn	code, and a library's private data

A call into another zone goes through a generated **veneer** that maps the target's zone, calls, and restores your mapping. It costs about a banked call (§7.7). The callee sees exactly the caller's arguments and stack, so any function can be reached this way. Data never crosses zones: every static and every `const` object, string literals included, is resident, so a pointer is valid from anywhere. A function whose address is taken must therefore be resident too. A module that maps far data itself is resident.

You tell the toolchain only what it cannot work out:

- `RESIDENT=` lists your event handlers' sources. A handler runs wherever the program was interrupted, far code included, so it and everything it calls directly must stay visible.
- `PLACE=` pins a module, for example `PLACE=view.c=far1` or `PLACE=libgfx.lib(gfx_text.rel)=home`. An SDK library places as one unit.

```
$(eval $(call exe_program,tracker,.,main.c:view.c:play.c,LIBS=snd:gfx,RESIDENT=play.c))
```

The rest is automatic: code stays resident while there is room, the most-called first, and the remainder is grouped along the call graph into the home zone and far segments. The result is deterministic. The link writes every module's zone and size, and every veneer, to `build/user/NAME/place/placement.txt`. A veneer called from an inner loop costs about 31 µs per call; a `PLACE=` pin that puts caller and callee in one zone removes it. The packer checks the zone rule from the relocations and names the module and symbol that broke it (Appendix C).

PART II · 8

I/O performance habits

The costs that matter are per **call**, not per byte. One console write costs about 15,000 T-states however few bytes it carries; one card read costs about 30,000 plus the block. A program that makes fewer, larger calls is faster by an order of magnitude.

8.1 The measured numbers

Measured with the SDK's I/O benchmark, `tests/bench/iobench.c`. The throughput column is the per-byte cost converted at 10 MHz.

PATTERN	T-STATES PER BYTE	ABOUT
`read` of 1 byte	29,000	340 B/s
`getc` on a buffered stream	2,600	3.8 KiB/s
`putchar` (one console call per 128-byte line)	1,330	7.3 KiB/s
`read` of 512 bytes	1,170	8.3 KiB/s

8.2 What this means for your program

- **Read in blocks of 512 bytes or more.** A byte-at-a-time `read` pays the full call overhead per byte, about 25 times the cost of a block read.
- **Use streams when you must consume bytes one at a time.** `getc` pays the overhead once per 512 bytes. For bulk data, a direct 512-byte `read` into your own buffer is faster still.
- **Let the console buffer work.** Avoid `console_flush` after every write; end lines with `\n` and the buffer flushes itself. For large output, one `console_write` of the whole block is a single call.
- **Plan sustained reads.** A program that must keep up with a fixed rate, such as a player, should measure the medium and size its pre-fetch rather than guess.

Chapter 10 shows how.

PART II · 9

Device events

A program can subscribe to events on a device handle it holds. The driver posts an event from its interrupt; the system delivers it by running a C function of yours at the next point where it is your program's turn to run. You write no interrupt code and no polling loop, and the system restores your program's exact state when the handler returns.

There are two event sources in the current build:

- **the event timer** on `/dev/timer`, armed with `timer_event` at the period you choose;
- **the V9958 frame interrupt**, on a `/dev/video` handle bound with `KOS_BIND_EXCLUSIVE`. The program enables the interrupt itself (IE0, R#1 bit 5) and receives one event per frame. `libgfx` keeps it off while it holds the display (§12.6).

A handle whose device serves no events answers `-KOS_ENOTSUP` when you subscribe.

9.1 A periodic tick

```
#include "kos.h"
#include "kos/timer.h"
#include "kos/event.h"

static volatile unsigned int ticks;
static volatile unsigned int lost;

static void on_tick(unsigned int count, unsigned int missed)
{
        ticks += count;
        lost += missed;
}

int main(void)
{
        int t = timer_open();

        if (t < 0)
                return -t;
        if (notify(t, on_tick) < 0)
                return 1;
        timer_event(t, 782);            /* 782 counts = 20 ms */
        while (ticks < 250)
                idle();                 /* five seconds, CPU halted */
        timer_event(t, 0);              /* stop */
        notify(t, 0);                   /* unsubscribe */
        printf("ticks %u missed %u\r\n", ticks, lost);
        return 0;
}
```

Subscribe with `notify` first: `timer_event` on a handle with no subscription is `-KOS_EINVAL`. A second `notify` on the same handle replaces the handler; `notify(handle, 0)` unsubscribes. Exit and `unbind` also end the subscription, and releasing the timer stops its event channel.

A handler runs in place of the interrupted code, so a 16-bit variable it updates can change between the two byte reads of a load in `main`. Where a torn value matters, read it twice and repeat until both reads agree.

9.2 When delivery happens

The system checks for a deliverable event at two points only, and only for the process it is returning to:

YOUR PROGRAM IS …	DELIVERY
running its own code, and an interrupt ends	now, before the interrupted instruction
inside one of its own system calls, and the call ends	now, before the call's result is returned
inside its own handler	held until the handler returns; handlers never nest
waiting on a child started with `run`	dropped, and counted in `missed`
exited, or has unbound the handle	the subscription is gone

9.3 `count` and `missed`

Events coalesce; nothing queues. `count` is how many events arrived since the last delivery and `missed` how many were dropped since then. Both saturate at `FFFFh`, and both restart at every delivery, so a program that wants running totals adds them up, as the example does.

A handler slower than its period does not fall behind a queue: it sees a larger `count` next time. Reading the clock inside the handler gives the exact time of delivery, which is the only delay between the deadline and your code.

9.4 Rules for handlers

- **It runs on your stack.** The system builds a 24-byte delivery frame below your stack pointer and runs the handler below that. Size `STACK` for your deepest call chain plus the frame plus the handler's own depth, including any SDK calls it makes.
- **Keep the stack out of the aperture.** The frame may not lie in `8000h-BFFFh`. The far link rule and the default stack placement already guarantee this.
- **In a placed program, make it resident.** Name the handler's source in `RESIDENT=`. Everything it calls directly becomes resident with it (§7.8).
- **It may call the SDK.** A handler can print, read the clock or call a driver. The SDK's trampoline saves its 93-byte argument area before the handler and restores it afterwards, so a handler cannot corrupt a request the interrupted code was in the middle of. The snapshot is two 93-byte copies per delivery, about 1,970 T-states each.

- **Return normally.** Never leave a handler by a long jump. The system restores the interrupted state only when the handler returns; one that never returns leaves the delivery open.
- **Leave far data alone.** Copy what the handler needs into primary memory beforehand, rather than assume what the aperture shows at the moment of delivery.

9.5 Timing and cost

Periods are given in clock counts of 25.6 µs, which is 390,625 counts per 10 s: 782 counts is 20 ms (50 Hz) and 19,531 is 500 ms. The driver keeps absolute deadlines and adds the period to each, so deadlines never drift. A stretch with interrupts masked can make a delivery late, but never lost, doubled or early. Measured without card traffic, deliveries sit within 1 count of the deadline at the median and 16 at worst.

PART OF ONE DELIVERY	T-STATES
frame build and hand-off	1,710
trampoline entry, with the argument-area snapshot	2,568
restore, return gate and unwind	4,129
total, before your handler's body	**8,407, or 4.2% of the CPU at 50 Hz**

9.6 Limits

- Four subscription slots; when they are taken, `notify` answers `-KOS_ENOSPC`.
- One subscription per process and handle.
- The event timer has one subscriber at a time; a second answers `-KOS_EBUSY`.

9.7 Raw sound output

```
#include "kos/sound.h"
int  sound_open(unsigned char exclusive);
int  sound_close(int handle);
int  sound_info(int handle, struct ksound_info *info);
int  sound_write(int handle, const struct ksound_write *writes,
                 unsigned int count);
int  sound_reset(int handle);
void sound_out(unsigned char port, unsigned char value);
```

`/dev/sound` is the sound card: four SN76489 PSGs and a DAC. This section is the register-level interface. For notes, envelopes and songs use `libsnd` (chapter 13), which is built on it. There are two ways to drive the card directly.

- **Batched writes.** Build an array of `{target, value}` pairs, target 0-3 a PSG and 4 the DAC, and send it with one `sound_write`. A call costs about 15,500 T-states, so one tick's register writes belong in one call. It returns the number of writes performed; an unknown target is `-KOS_EINVAL` with nothing from that call written. A sequence over 4 KiB is split into several calls for you.
- **Direct I/O.** `sound_open(1)` takes the card exclusively and resets it; a second exclusive open is `-KOS_EBUSY`. Read the ports and the PSG clock from `sound_info`, never hard-code them, and write them with `sound_out`, which costs no call per byte.

```
static const struct ksound_write a440[] = {
        { 0, 0x8E },   /* PSG 0: channel 0 tone, low 4 bits  */
        { 0, 0x0F },   /* tone, high 6 bits: N = 0FEh, 440 Hz */
        { 0, 0x90 },   /* channel 0 volume, loudest          */
};

int s = sound_open(0);

if (s ⩾ 0)
        sound_write(s, a440, 3);
```

Timing is yours: a write happens when it is sent. Pace music with a timer event (§9.1) and send each tick's writes in one call. `sound_reset` silences the card on any handle, and `sound_close` or exit resets it, so the next program never inherits a sounding PSG. A build without the sound driver answers `-KOS_ENOENT` or `-`

`KOS_ENODEV` from `sound_open`.

9.8 A reference program

`/system/beat.exe` in the production store subscribes at 500 ms, sounds a tone at every delivery for ten seconds and prints `count/missed` with the clock reading once a second. `beat run <seconds>` changes the length and `beat ls <dir> <seconds>` runs it while listing a directory, which adds card traffic. It is the quickest way to confirm that events are delivered on your board, and its source, `tests/beat/beat.c`, is a complete worked example.

PART II · 10

Streams, measurement and planning

The SDK's buffered file layer is a small ISO C subset with no allocation. On top of it, `kos/stream.h` lets a program time its own reads and work out, before it starts, whether the medium can feed it at the rate it needs.

10.1 The pool

There are four streams, each with a 512-byte buffer, in one fixed pool of about 2.2 KiB of BSS. A program that calls `fopen` carries the whole pool however many files it opens; one that never calls it carries none of it. `fopen` takes a free stream and returns `NULL` when the pool is full or the open failed. Modes are `"r"`, `"w"` and `"a"`; a `'b'` is accepted.

The card serves only two open files at once, so on `/fs` the third `fopen` fails even with streams free. Close what you no longer need.

10.2 How the buffer behaves

OPERATION	BEHAVIOUR
reading	one card read refills the 512-byte buffer
writing	bytes leave when the buffer fills, at `fflush`, at `fclose` and at exit
a transfer of 512 bytes or more	bypasses the buffer and moves the bytes once, straight to or from your memory
`fseek`	flushes pending writes and drops the read buffer; the next read starts where it says
`ftell`	the logical position, counting bytes still in the buffer
`fflush(NULL)`	flushes the console and every open stream
exit	flushes every stream, through the console flush

`feof` and `ferror` report a stream's state. A call that cannot report an error through its ISO result returns the negated native code, so `-KOS_EMFILE` and the rest still

reach you.

10.3 Choosing an interface

YOU NEED TO	USE
parse text a byte or a line at a time	`getc`, `fgets`
move bulk data	`fread`/`fwrite` of 512 bytes or more, or `read`/`write`
read small records at known offsets	`fseek` and `fread`
feed a consumer at a fixed rate	a stream with `fmeasure`, and `stream_plan`

10.4 Measuring refills

```
#include "kos/stream.h"
int fmeasure(FILE *stream, int timer);   /* timer from timer_open() */
int fstats(FILE *stream, struct kos_stream_stats *stats);
```

A stream never needs the clock unless you ask. `fmeasure` opts one stream in; from then on every refill, the read that fills its buffer, is timed. `fstats` reports how many refills there were and the fastest, mean and slowest, in 25.6 µs counts. Measure on the card and the file you will actually use: refill times vary with the medium.

10.5 Planning a sustained read

`stream_plan` turns a consumer's needs and the refill times, measured or brought from elsewhere, into a schedule:

```
struct kos_stream_need {            /* in */
    unsigned long rate;    /* B: bytes per second the consumer needs  */
    unsigned int  period;  /* T: the tick it is served within, counts */
    unsigned int  chunk;   /* C: the refill size, bytes (512)         */
    unsigned int  mean;    /* t_mean: mean refill time, counts        */
    unsigned int  worst;   /* t_worst: worst refill time, counts      */
    unsigned int  safety;  /* pre-fetch factor; 0 selects 2           */
};
struct kos_stream_plan {            /* out */
    unsigned long bytes_per_tick_millis;  /* b = B*T, milli-bytes     */
    unsigned int  ticks_per_refill;       /* n = floor(C / b)         */
    unsigned long prefetch;               /* P = safety*B*t_worst + C */
    unsigned long startup_ms;             /* P / R, R = C / t_mean    */
    unsigned int  utilisation_pct;        /* U = B / R                */
    unsigned char feasible;               /* U < 50 %                 */
};
int stream_plan(const struct kos_stream_need *need,
                struct kos_stream_plan *plan);
```

Read the plan as: consume `b` bytes per tick for `n` ticks, then refill with one `C`-byte read right after the tick. A refill that runs longer than `T` makes the next tick late, never lost. `P` is what must be in hand before the consumer starts, so that even the worst refill never starves it; `startup_ms` is how long collecting `P` takes at the mean rate. A plan with `feasible` 0 is not an error. It says this medium cannot serve this consumer at the safety you asked for.

Worked against a card with 512-byte reads at 18.5 ms mean and 150 ms worst, and a 20 ms tick (782 counts):

CONSUMER	B (MILLI-B/TICK)	N	P	U	START-UP
100 KiB over 180 s (570 B/s)	11,370	45	1,024 B (2 chunks)	2%	37 ms
100 KiB over 30 s (3,413 B/s)	68,325	7	1,536 B (3 chunks)	12%	55 ms
20,000 B/s	–	–	6,656 B	72%	not feasible

`stream_plan` does its arithmetic without the timer and links on its own, so a program can plan with figures it brings without opening a stream.

10.6 Putting events and streams together

A player, or any consumer that must be fed at a steady rate, combines this chapter with the previous one:

1. Open the file with `fopen`, call `fmeasure`, and read a few chunks to calibrate.
2. Take the mean and worst refill times from `fstats`, fill in a `kos_stream_need` with your rate and tick, and call `stream_plan`. If `feasible` is 0, lower the rate or tell the user.
3. Read until `prefetch` bytes are in hand.
4. Subscribe to the timer with `notify` and start `timer_event` at the plan's period. In the handler, add `count` to a tick total.
5. In the main loop, consume `b` bytes per tick, and after every `n` ticks make one `C`-byte refill straight after the tick.
6. Watch `missed`. It should stay 0; a growing value means the main loop is not keeping up.

PART II · 11

Text user interfaces

`libtui` is the SDK's optional text-screen layer. It keeps a cell map of the console's grid, decodes the console's key sequences into key values, and sends changes only when you flush. `libtui_widgets` is a second optional archive of procedural controls built on it. Your program never writes escape sequences or box-drawing bytes, and a program that links neither library pays nothing for them.

11.1 A first screen

```
#include "kos.h"
#include "tui.h"

static struct tui_size size;

static void draw(void)
{
        struct tui_rect body;

        body.x = 0;
        body.y = 1;
        body.w = size.cols;
        body.h = size.rows - 2;
        tui_begin();
        tui_set_style(TUI_STYLE_TITLE);
        tui_puts_xy(0, 0, " Kona Utility ");
        tui_frame_title(&body, TUI_FRAME_SINGLE, TUI_STYLE_TITLE, "Panel");
        tui_set_style(TUI_STYLE_NORMAL);
        tui_puts_xy(2, 2, "F10 or ESC quits");
        tui_set_style(TUI_STYLE_STATUS);
        tui_puts_xy(0, size.rows - 1, "Ready");
        tui_end();                      /* one flush for the whole screen */
}

int main(void)
{
        tui_key_t key;

        if (tui_init() < 0 || tui_get_size(&size) < 0)
                return 1;
        draw();
        do
                key = tui_getkey();     /* idles between empty polls */
        while (key != TUI_KEY_F10 && key != TUI_KEY_ESC);
        tui_shutdown();
        return 0;
}
```

```
$(eval $(call exe_program,my_ui,.,my_ui.c,STACK=512,LIBS=tui))
```

The screen size comes from the console, not from a constant. Every drawing call between `tui_begin` and `tui_end` only changes the cell map; the screen changes once, at `tui_end`.

11.2 Linking and capacity

YOU WANT	BUILD WITH	INCLUDE
screen, keys, windows, frames	`LIBS=tui`	`tui.h`
menus, buttons, inputs, dialogs	`LIBS=tui_widgets:tui`, widgets first because they depend on the core	`tui_widgets.h`

The capacity is a build option. `TUI_GRID=85x26`, the default, fits both of today's consoles; `TUI_GRID=128x35` links the larger library. At run time the library uses the grid the console reports, clamped to the capacity, so on a larger console the program draws in the top-left part. Choose the largest grid you want to use, since capacity costs static storage:

CAPACITY	CHARACTERS	ATTRIBUTES	DIRTY AND INVERT BITMAPS	RUN BUFFER
85 × 26	2,210	1,105	2 × 277	101
128 × 35	4,480	2,240	2 × 560	144

That is about 4 KiB or 8 KiB of BSS, plus a small state block. The formatter and each optional group of functions are separate members. The widget archive's modal member owns one hidden 1,920-byte save buffer and links only if you use a message box or confirmation. There is no heap: save buffers and widget lists are yours. The SDK's examples build with `STACK=512`.

11.3 Lifecycle

`tui_init()` puts the terminal and the model in agreement: normal rendition, a blank cell map, the cursor home and visible, no automatic wrapping. A second `tui_init()` while initialised returns `-KOS_EINVAL` and leaves the screen alone. `tui_shutdown()` flushes pending cells as best it can, restores normal rendition, wrapping and the cursor, and is safe to call twice. Call it on every normal exit path; a program that exits without it may leave the terminal in its last state.

Before `tui_init()`, and after shutdown until the next init, drawing, flushing and `tui_key_pending` answer `-KOS_EINVAL`, and `tui_pollkey` and `tui_getkey` return `TUI_KEY_NONE` without spinning.

11.4 Coordinates and windows

Coordinates are 0-based, and the origin is the top-left of the **active window**, not necessarily the screen. Drawing at the right edge is clipped, not wrapped, and there is no implicit scrolling.

```
tui_window(x, y, w, h);     /* absolute rectangle, relative drawing */
tui_reset_window();         /* the full screen */
tui_push_window(&rect);     /* nested, at most four deep */
tui_pop_window();
tui_get_window(&rect);      /* the active window, absolute */
```

A window must be non-empty and lie fully on the screen. Installing or popping one homes the cursor. Overflow and underflow of the stack return `-KOS_EINVAL` without changing anything. Every text, clear, line and drawing call clips to the active window, so `tui_clear`, `tui_clreol`, `tui_clrline`, `tui_insert_line` and `tui_delete_line` affect only that window.

Text goes through `tui_putc`, `tui_puts`, `tui_write` and `tui_printf`, which advance the cursor, and their `_xy` forms, which leave it alone. `tui_put_text(x, y, glyphs, length, attr)` stores a literal row instead: every byte as itself, with one attribute, without moving the cursor, clipped at the window's right edge. In the other calls, control bytes are interpreted inside the window:

BYTE	EFFECT
BS (`08h`)	left one column, stopping at column 0
TAB (`09h`)	to the next eight-column stop, stopping at the last column
LF (`0Ah`)	down one row, stopping at the last row; the column is kept
CR (`0Dh`)	to column 0
other control bytes	ignored

`tui_scroll_rows(top, height, TUI_SCROLL_UP)`, or `TUI_SCROLL_DOWN`, moves an absolute, full-width region of the screen by one row and blanks the exposed row. It flushes pending changes first, keeps the window and the cursor, and shifts the cell map without marking the retained rows dirty; on the V9958 the console moves the display origin rather than redrawing the region. A console without scroll support, such as the serial console, answers `-KOS_ENOTSUP` and changes nothing, so repaint the region instead.

11.5 Drawing is deferred

Drawing changes the cell map and marks cells dirty; nothing reaches the console until a flush. `tui_begin` and `tui_end` nest, and only the outermost `tui_end` flushes; an unmatched `tui_end` is `-KOS_EINVAL`. `tui_flush` always flushes, even inside a transaction.

A flush sends only what changed. On the V9958 console it is one render transaction, and a change of normal and reverse video alone is done in place, so moving a selection does not redraw its labels. A console without the render extension, such as the serial console, answers `-KOS_ENOTSUP` once; the library remembers that and sends ANSI runs instead. A redraw that changes no cell and no cursor state sends nothing. If a flush fails, cells it did not commit stay pending for the next one.

11.6 Styles and frames

Draw with semantic styles so your code does not depend on the console's colours: `NORMAL`, `TITLE`, `SELECTED`, `DISABLED`, `ERROR`, `INPUT`, `STATUS` and `ACCENT`, each `TUI_STYLE_*`. Today `TITLE`, `SELECTED` and `STATUS` show as reverse video and the rest as normal. `tui_set_style` affects later drawing only. `tui_set_attr` is the low-level escape and accepts only 0 (normal) or 1 (reverse).

```
tui_fill(&rect, ' ', TUI_STYLE_NORMAL);
tui_hline(x, y, width, TUI_FRAME_SINGLE);
tui_vline(x, y, height, TUI_FRAME_DOUBLE);
tui_frame(&rect, TUI_FRAME_SINGLE, TUI_STYLE_TITLE);
tui_frame_title(&rect, TUI_FRAME_DOUBLE, TUI_STYLE_TITLE, "Panel");
```

Frames handle widths and heights of 1 and 2 without doubled corners. A title is clipped inside the top edge, with a blank either side where there is room, and never written outside the rectangle.

11.7 Keys

```
int       pending = tui_key_pending(); /* >0 ready, 0 empty, <0 error */
tui_key_t key = tui_pollkey();         /* never blocks; NONE if empty */
tui_key_t key = tui_getkey();          /* idles until a key is ready  */
```

Ordinary bytes keep their value, including CR, TAB, BS and DEL. Special keys have values above `00FFh`:

KEY	VALUE
`TUI_KEY_ESC`	`001Bh`
`UP`, `DOWN`, `LEFT`, `RIGHT`	`0101h`, `0102h`, `0103h`, `0104h`
`HOME`, `END`	`0110h`, `0111h`
`PGUP`, `PGDN`	`0112h`, `0113h`
`INSERT`, `DELETE`	`0114h`, `0115h`
`F1` … `F12`	`0121h` … `012Ch`

A lone ESC is returned at once; the library does not wait for a following byte. An unknown or malformed sequence returns ESC and then replays the rest as ordinary input, so no keystroke is lost.

11.8 Saving and restoring a region

```
struct tui_rect box = { 10, 5, 20, 8 };
uint16_t need = tui_saved_size(&box);     /* bytes */

if (tui_save(&box, my_storage, need) < 0)
        return 1;
/* draw the overlay */
tui_restore(&box, my_storage, need);
```

The rectangle is relative to the active window and clipped once. You supply the storage, sized by `tui_saved_size`; a short or null buffer is refused before anything is copied. Restore marks only changed cells dirty and does not move the cursor, change the style or window, or flush.

11.9 Widgets

```
tui_statusbar("status");                  /* bottom row of the window */
tui_menubar(items, count, &selected);     /* top row */
tui_label(x, y, "label");
tui_button(x, y, "OK", TUI_ID_OK, focused);
tui_input(x, y, width, buffer, buffer_size);
tui_checkbox(x, y, "Enabled", &checked);
tui_listbox(&rect, &list);
tui_id_t id = tui_menu_run(items, count);
int ok = tui_input_run(x, y, width, buffer, buffer_size);
tui_id_t answer = tui_message_box(title, message, buttons);
tui_id_t answer = tui_confirm(title, message);
```

Widgets are procedural functions with no object model, and their coordinates are relative to the active window. The render-only calls draw and leave input handling to you (§11.10). The `_run` calls and the dialogs read keys themselves:

- `tui_menu_run` starts on the first item and returns the chosen item's ID, or `TUI_ID_NONE` on ESC.
- `tui_input_run` edits with printable keys, arrows, HOME, END, BS and DELETE, and returns 1 on ENTER, 0 on ESC or a negative error.
- Message boxes take a mask built from `TUI_BUTTON(TUI_ID_OK)` and the `CANCEL`, `YES` and `NO` IDs. Buttons appear in the order OK, YES, NO, CANCEL. LEFT, RIGHT or TAB move focus, ENTER chooses, and ESC chooses CANCEL when present, otherwise `TUI_ID_NONE`. `tui_confirm` is YES/NO with YES focused.
- Dialogs centre in the active window, wrap at word boundaries, are at most 60 × 16, and restore exactly what they covered. A nested dialog call is refused with `TUI_ID_NONE`.

11.10 Reacting to input

Widgets come in two kinds, and you react to each differently:

KIND	CALLS	HOW YOU GET THE USER'S CHOICE
modal: reads keys itself and returns	`tui_menu_run`, `tui_input_run`, `tui_message_box`, `tui_confirm`	from the return value
render-only: draws from your variables	`tui_button`, `tui_checkbox`, `tui_listbox`, `tui_menubar`, `tui_input`, `tui_label`, `tui_statusbar`	from your own variables, which your key loop changes

Menus and dialogs: switch on the result

```
switch (tui_menu_run(items, 3)) {
case ID_OPEN:
        open_file();
        break;
case ID_QUIT:
        if (tui_confirm("Quit", "Leave the program?") == TUI_ID_YES)
                done = 1;
        break;
case TUI_ID_NONE:               /* ESC: nothing chosen */
        break;
}
```

A message box works the same way: `tui_message_box("Save", "Overwrite the file?", TUI_BUTTON(TUI_ID_YES) | TUI_BUTTON(TUI_ID_NO) | TUI_BUTTON(TUI_ID_CANCEL))` returns the ID of the button chosen, `TUI_ID_CANCEL` on ESC because Cancel is present.

A form: state, a key loop and a redraw

For a form of checkboxes, a list and buttons, the program owns all the state: the value of each checkbox, the list's `selected` and `top`, and which control has focus. The loop is always the same: draw everything from the state, read one key, change the state, repeat. Redrawing the whole form after every key is cheap, because a flush sends only the cells that changed (§11.5).

```
#include "kos.h"
#include "tui_widgets.h"

enum { F_ENABLED, F_VERBOSE, F_LIST, F_OK, F_CANCEL, F_COUNT };

static const char * const ports[] = {
        "Serial 0", "Serial 1", "Card", "Sound"
};
static struct tui_list list;           /* zeroed: set in main */
static struct tui_rect box;
static uint8_t enabled, verbose, focus;

static void draw(void)
{
        tui_begin();
        tui_putc_xy(0, 1, focus == F_ENABLED ? '>' : ' ');
        tui_checkbox(2, 1, "Enabled", &enabled);
        tui_putc_xy(0, 2, focus == F_VERBOSE ? '>' : ' ');
        tui_checkbox(2, 2, "Verbose", &verbose);
        tui_putc_xy(0, 4, focus == F_LIST ? '>' : ' ');
        tui_listbox(&box, &list);
        tui_button(2, 9, "OK", TUI_ID_OK, focus == F_OK);
        tui_button(10, 9, "Cancel", TUI_ID_CANCEL, focus == F_CANCEL);
        tui_end();
}
```

The key handler turns one key into a state change. It returns `TUI_ID_OK` or `TUI_ID_CANCEL` when the form is finished, and `TUI_ID_NONE` to keep going:

```
static tui_id_t handle(tui_key_t key)
{
        if (key == TUI_KEY_ESC)
                return TUI_ID_CANCEL;
        if (key == '\t') {                      /* TAB: next control */
                if (++focus == F_COUNT)
                        focus = 0;
                return TUI_ID_NONE;
        }
        switch (focus) {
        case F_ENABLED:
                if (key == ' ' || key == '\r')
                        enabled = !enabled;     /* toggle the box */
                break;
        case F_VERBOSE:
                if (key == ' ' || key == '\r')
                        verbose = !verbose;
                break;
        case F_LIST:
                if (key == TUI_KEY_UP && list.selected > 0)
                        list.selected--;
                if (key == TUI_KEY_DOWN && list.selected + 1 < list.count)
                        list.selected++;
                if (list.selected < list.top)        /* keep it visible */
                        list.top = list.selected;
                if (list.selected >= list.top + box.h)
                        list.top = list.selected - box.h + 1;
                break;
        case F_OK:
                if (key == '\r')
                        return TUI_ID_OK;       /* the button was pushed */
                break;
        case F_CANCEL:
                if (key == '\r')
                        return TUI_ID_CANCEL;
                break;
        }
        return TUI_ID_NONE;
}
```

```
int main(void)
{
        tui_id_t r;

        if (tui_init() < 0)
                return 1;
        list.items = ports;
        list.count = 4;
        box.x = 2;
        box.y = 4;
        box.w = 20;
        box.h = 4;
        do {
                draw();
                r = handle(tui_getkey());
        } while (r == TUI_ID_NONE);
        tui_shutdown();
        if (r == TUI_ID_OK)
                printf("enabled %u verbose %u port %s\r\n",
                       enabled, verbose, ports[list.selected]);
        return 0;
}
```

After the loop, the answers are simply the variables: `enabled` and `verbose` are the checkboxes, `list.selected` is the index of the chosen list item, and `r` says which button ended the form. Keys arrive as `'\r'` for ENTER, `'\t'` for TAB and `' '` for space; special keys are the `TUI_KEY_*` values of §11.7.

- **A menu bar** follows the same pattern: `selected` holds an item *ID*, not an index. Move it on LEFT and RIGHT, draw with `tui_menubar`, and act on ENTER with the same `switch` as for `tui_menu_run`.
- **A text field** is drawn with `tui_input` while it does not have focus. When the user presses ENTER on it, hand over to `tui_input_run` with the same position, width and buffer; it returns 1 when the user commits with ENTER and 0 on ESC. Keep a copy of the buffer first and put it back on 0 if ESC should discard the edit.
- **Showing focus** is your choice for controls without a `focused` argument. The example puts a marker in column 0; a `TUI_STYLE_SELECTED` label works too.

11.11 Results

Unless stated otherwise, calls return 0 or a negated `KOS_E*` code; a null required pointer, a bad rectangle or style, a call before `tui_init`, a window-stack overflow or an unmatched `tui_end` is `-KOS_EINVAL`. `tui_putc` returns 1, the string writers return the bytes they were given even when clipping hid some of them, and the formatters return the formatted length. The SDK's `tui_demo`, `tui_keys` and `tui_dialog` examples are complete programs; §1.6 walks through a short menu.

PART II · 12

Graphics

`libgfx` draws on the V9958's bitmap modes. It takes `/dev/video` exclusively and then drives the chip itself, so no drawing call crosses into the system, and every routine that draws or moves pixels is assembly. It is a drawing library, not a GUI toolkit: there are no windows, controls or layout. A program that does not link it pays nothing for it.

12.1 A first picture

```
#include "kos.h"
#include "kos/gfx.h"

static struct gfx_text_style style;    /* zeroed: opaque 6x8 */

int main(void)
{
        if (gfx_open() < 0)
                return 1;
        if (gfx_set_mode(GFX_MODE_512x212_16) < 0) {
                gfx_close();
                return 1;
        }
        gfx_clear(0);
        gfx_fill_rect(16, 16, 200, 40, 4, GFX_OP_COPY);
        gfx_rect(16, 16, 200, 40, 15, GFX_OP_COPY);
        style.fg = 15;
        style.bg = 4;
        gfx_text(24, 32, "Hello, KonaOS", &style);
        gfx_display_enable(1);
        getchar();                      /* wait for a key */
        gfx_close();                    /* the console comes back */
        return 0;
}
```

```
$(eval $(call exe_program,picture,.,picture.c,LIBS=gfx))
```

Colours are palette indices. While the program holds the display the text console is not on screen; `gfx_close` releases it and the system restores the console. Exit does the same if the program never calls it, so a failed graphics program never needs a reboot.

12.2 Opening and linking

`gfx_open` binds `/dev/video` with `KOS_BIND_EXCLUSIVE`, asks the driver for the direct-access information only an exclusive holder receives, and sets up the library's one process-local context. It does not change the display. If it fails after taking the device it releases it again, so a failed open leaves nothing behind. `gfx_set_mode` then sets a complete known state, whatever the console left in the VDP, and `gfx_get_info` reports the mode, the size and the page shown.

Build with `LIBS=gfx` and include `kos/gfx.h`. Each group of calls is its own member, and the V9958 port code links only through the drawing calls.

12.3 Modes and coordinates

MODE	V9958	PIXELS	USE IT FOR
`GFX_MODE_512x212_16`	GRAPHIC 6	16-colour palette, two pixels per byte, the left one in the high nibble	interfaces, editors, dense text
`GFX_MODE_256x212_256`	GRAPHIC 7	one byte per pixel: green in bits 7-5, red 4-2, blue 1-0	pictures
`GFX_MODE_256x212_YJK`	GRAPHIC 7 with YJK	one host-encoded YJK byte per pixel	photographs

Every mode is 212 lines, non-interlaced. Coordinates are absolute in VRAM: y runs 0-511, lines 0-211 are page 0 and lines 256-467 page 1, and each page has 44 spare lines below its visible ones (212-255 and 468-511). All of it belongs to your program while it holds the display: keep tiles, sprites and icons off screen and copy them into place.

In the 16-colour mode two pixels share a byte, so every x and every width passed to a drawing, upload or text call must be even; an odd one is `-KOS_EINVAL`. The 256-pixel modes take any x. YJK shares colour across runs of four pixels, so the only drawing it accepts is a `COPY` fill aligned to four pixels; other shapes and text

answer `-KOS_ENOTSUP`.

12.4 Drawing

```
gfx_clear(color);
gfx_fill_rect(x, y, w, h, color, GFX_OP_COPY);
gfx_rect(x, y, w, h, color, GFX_OP_XOR);
gfx_line(x0, y0, x1, y1, color, GFX_OP_COPY);
gfx_point(x, y, color, GFX_OP_COPY);
gfx_ellipse(cx, cy, rx, ry, color, GFX_OP_COPY);
gfx_fill_ellipse(cx, cy, rx, ry, color, GFX_OP_COPY);
gfx_copy_rect(src_x, src_y, dst_x, dst_y, w, h);
```

The clear and the copy always overwrite. Every shape takes an operation: `GFX_OP_COPY` writes the colour, and `GFX_OP_XOR` combines it with what is there, so drawing the same shape twice restores the screen. That is the cheap way to draw a cursor, a play marker or a selection. Rectangles plot each corner once and an ellipse outline never overlaps itself, so XOR is exact. A circle is an ellipse with `rx == ry`.

`gfx_copy_rect` picks a safe direction when the rectangles overlap, and either side may be on the off-screen page. A shape that does not lie wholly inside the mode's width and the 512-line space is `-KOS_EINVAL` and draws nothing.

12.5 Drawing runs alongside your program

The VDP's command engine does the drawing. libgfx waits for the engine before it starts a command, not after, so a drawing call returns while the VDP is still busy and your program carries on. Every call that touches VRAM through the CPU (an upload, opaque text, the palette, a mode set, a page switch, `gfx_close`) waits for the engine first, so it never races a command. Call `gfx_wait()` when you need the drawing finished, for example to time a frame.

`gfx_begin` and `gfx_end` queue commands instead of issuing them, up to 32, flushed in order when the queue fills; `gfx_flush` sends them early. A queued command is built exactly as an immediate one, so a batch costs a copy per command and buys only the deferral. Use it when a group of commands should start together, not for speed. Commands are never reordered.

12.6 Pages and the vertical blank

```
int gfx_show_page(uint8_t page);    /* 0 or 1, at the next vertical blank */
int gfx_wait_vblank(void);          /* until the next retrace starts */
```

Both modes hold two pages. Draw the hidden one by its coordinates (y + 256 for page 1) and call `gfx_show_page`: it waits for the engine and the retrace and switches atomically, so there is no tearing.

- **Part of the screen.** Draw the band on the hidden page, call `gfx_wait_vblank()` and `gfx_copy_rect` it onto the shown page. The blank lasts about 3.2 ms and holds a copy of about two full-width 16-colour rows; a larger band still works, with a short tear.
- **Small items that change often**, such as meters, counters or a scope, are cheaper drawn in place on the shown page right after `gfx_wait_vblank()`. Copy them to the hidden page before the next switch so both pages agree.

The blank is found by polling the VDP. libgfx keeps the frame interrupt off while it holds the display, so a graphics program receives no frame events (chapter 9); pace a frame loop with `gfx_wait_vblank` or a timer event.

12.7 Pictures and uploads

```
gfx_upload_begin(x, y, width, height);
while ((n = read(file, buffer, sizeof buffer)) > 0)
        gfx_upload_write(buffer, n);
gfx_upload_end();
```

An upload streams a rectangle's pixels straight to VRAM in the mode's packing (§12.3). Chunks may be any length, so a picture never has to be in memory whole: feed it from a file, from extended memory through a small buffer, or from data you generate. The upload must receive exactly the bytes its rectangle needs, cannot be nested, flushes any pending batch first, and may target the off-screen page.

Pictures are prepared on the host. `tools/kimg.py` in the SDK, which needs Pillow, converts PNG, JPEG and anything else Pillow reads into `KIMG` v1 (`kos/kimg.h`): a 16-byte header with the mode, the size and a palette count, the palette

as RGB triples, then the payload ready to upload. It scales, crops, reduces colours and encodes YJK, so the Z80 does none of that. A full screen is 54,272 bytes in the 16-colour and the 256-colour mode alike.

`gfx_set_palette(first, count, colors)` programs the 16-colour palette from `struct gfx_rgb` entries, converted to the V9958's format for you. A mode without a palette answers `-KOS_ENOTSUP`.

12.8 Text

```
struct gfx_text_style {
        gfx_color_t fg;
        gfx_color_t bg;
        uint8_t flags;      /* GFX_TEXT_TRANSPARENT */
        uint8_t font;       /* GFX_FONT_6X8 (0) or GFX_FONT_4X6 */
};
int gfx_text(x, y, text, &style);
int gfx_text_n(x, y, text, length, &style);
uint16_t gfx_text_width(text, font);   /* length x 6 or x 4 */
```

Text needs no setup: after `gfx_set_mode`, `gfx_text` draws. There are two built-in CP850 fonts, 6 × 8 and 4 × 6; the smaller fits 128 columns by 35 rows in the 16-colour mode, and its tables link only if you use it. A zeroed style is opaque 6 × 8. A string is one line of bytes: newlines, tabs, escapes and UTF-8 are not interpreted.

Opaque text is written by the CPU straight into VRAM. **Transparent text**, which leaves the pixels behind the glyphs untouched, is copied by the VDP from a glyph atlas, and needs a band of VRAM first:

```
/* a band at y 212, 44 lines, for glyphs 32-127 */
gfx_text_atlas(GFX_FONT_6X8, 212, 44, 32, 96);
```

The band must lie wholly in one page's spare lines, and it is the only VRAM libgfx uses. One atlas per text colour is built on that colour's first transparent use; when the band is full, the least recently used is rebuilt. In the 16-colour mode, printable ASCII takes 16 lines per colour in the 6 × 8 font and 6 in the 4 × 6. Transparent text without an atlas is `-KOS_EINVAL`, and a glyph outside the atlas range draws nothing.

Glyphs can be redefined, for symbols such as play, stop or an arrow in codes above `7Fh`. `gfx_font_set_glyph(font, code, rows)` takes 8 rows for 6 × 8 or 6 for 4 × 6, right-aligned, with the leftmost pixel in the highest bit used. `gfx_font_reset_glyph` and `gfx_font_reset` restore the built-in shapes. A redefined glyph is redrawn into every atlas at its next use.

12.9 Costs

CPU time at 10 MHz in the 16-colour mode. The VDP's own drawing continues after the call returns (§12.5).

OPERATION	T-STATES
one `gfx_fill_rect`	about 1,850
one `gfx_line`	about 1,700
a filled circle of radius 50; its outline	about 274,000; about 315,000
8 opaque 6 × 8 glyphs	about 12,300
a full row of 128 glyphs in 4 × 6	about 76,000, 590 per glyph
one call of 4 transparent glyphs	about 8,200
building a 95-glyph atlas, once per colour	about 73,000
the 4 × 6 font's first use, once	about 75,000

12.10 Programs to read

`gfxdemo` (`UserSoftware/GfxDemo`) uses the whole surface in one picture: a clear, bars, copied fills, text in two colour pairs and an application glyph drawn, restored and drawn again. `pview` (`UserSoftware/PView`) shows a `KIMG` file: it checks the header, opens the display, sets the mode, turns the display off, sets the palette, uploads, turns the display on, waits for a key and closes.

PART II · 13

Music and sound

`libsnd` plays music on the sound card's four PSGs. It does three things: notes and short phrases written in C, songs in the tracker's ZTR format, and pre-rendered ZVG streams. Everything that runs per tick is assembly; setup, parsing and file reads are C and run in your main loop. It is built on the raw interface of §9.7 and holds the card exclusively. It does not play samples on the DAC.

13.1 Ownership and the tick

```
#include "kos/snd.h"
int  snd_open(void);                    /* takes the card exclusively */
void snd_close(void);                   /* stop, silence, release */
int  snd_set_rate(uint16_t ticks_per_sec);
int  snd_timer_period(uint32_t *counts);
int  snd_tick(uint16_t elapsed_ticks);  /* from your timer handler */
int  snd_service(void);                 /* from your main loop */
```

libsnd keeps no clock of its own. Your program opens the timer, subscribes a handler and arms the event timer (chapter 9). `snd_timer_period` gives the period for the current rate: the song's while one plays, otherwise the one set with `snd_set_rate`. The handler passes its `count` to `snd_tick`, which advances the music and writes the registers that changed.

- **Late ticks are caught up, within a limit.** One call advances each voice by at most 16 ticks. The excess is counted as an overrun and never replayed, so a late delivery never causes a burst.
- **Reading belongs in the main loop.** Songs and streams are read by `snd_service`, which may call the file system. Call it from your main loop, never from the handler.
- **Size the stack** for the handler's depth, which now includes `snd_tick`.
- **In a placed program**, list the handler's source in `RESIDENT=`. libsnd's tick path

then becomes resident and its open and refill code does not (§7.8).

- **A child stops the music.** While a program started with `run` executes, your events are dropped (§9.2).

A program that only plays ZTR songs calls `snd_tick_ztr` and `snd_ztr_service` instead. They do the same work without the note engine and the ZVG decoder, which then do not link.

`snd_set_output(SND_OUTPUT_DIRECT)` writes the ports itself under the exclusive hold; `SND_OUTPUT_BATCH` sends batched `sound_write` calls. Both write the same registers in the same order. `snd_close` silences every voice and releases the card, and exit does the same.

13.2 Notes in C

The card has 16 voices, numbered by chip: on chip *n*, voices 4*n* to 4*n*+2 are tone voices and 4*n*+3 is the noise voice. Notes are numbered 0 (C-0) to 95 (B-7), with C-4 as 48. An instrument is a simple envelope:

```
typedef struct snd_instrument_simple {
        uint8_t  kind;            /* SND_INSTR_TONE or NOISE */
        uint8_t  peak_level;      /* 0..255 */
        uint16_t gate_ticks;      /* default duration */
        uint8_t  attack_ticks;
        uint8_t  decay_ticks;
        uint8_t  sustain_level;   /* 0..255 */
        uint8_t  release_ticks;
        uint8_t  noise_mode;      /* periodic or white */
        uint8_t  noise_rate;      /* 0..3; 3 = tone voice 2 */
} snd_instrument_simple_t;

int snd_define_instrument(uint8_t id, const snd_instrument_simple_t *i);
int snd_note_on(snd_voice_t voice, snd_note_t note, uint8_t instrument);
int snd_note_for(snd_voice_t voice, snd_note_t note,
                 uint16_t duration_ticks, uint8_t instrument);
int snd_note_off(snd_voice_t voice);      /* start the release */
int snd_voice_silence(snd_voice_t voice);
int snd_set_noise(snd_voice_t noise_voice, uint8_t mode, uint8_t rate);
```

A note call compiles the note and its envelope into a short program of register values and waits, so the tick only steps through prepared bytes. Notes go to tone

voices; `snd_set_noise` configures a noise voice. Noise rate 3 takes its pitch from tone voice 2 of the same chip, which is then occupied. Pitches come from a built-in A440 table for the card's 3,579,545 Hz clock.

```
#include "kos.h"
#include "kos/timer.h"
#include "kos/event.h"
#include "kos/snd.h"

static volatile unsigned int ticks;
static snd_instrument_simple_t bell;   /* zeroed: set in main */

static void on_tick(unsigned int count, unsigned int missed)
{
        snd_tick(count);
        ticks += count + missed;
}

int main(void)
{
        uint32_t period;
        int t;

        if (snd_open() < 0)
                return 1;
        snd_set_rate(50);
        bell.kind = SND_INSTR_TONE;
        bell.peak_level = 190;
        bell.gate_ticks = 8;
        bell.decay_ticks = 2;
        bell.sustain_level = 90;
        bell.release_ticks = 3;
        snd_define_instrument(0, &bell);
        t = timer_open();
        if (t < 0 || snd_timer_period(&period) < 0
            || notify(t, on_tick) < 0) {
                snd_close();
                return 1;
        }
        timer_event(t, period);
        snd_note_for(0, SND_C4, 8, 0);         /* a C major chord */
        snd_note_for(1, SND_E4, 8, 0);
        snd_note_for(2, SND_G4, 8, 0);
        while (ticks < 50)                      /* one second */
                idle();
        timer_event(t, 0);
        notify(t, 0);
        snd_close();
        return 0;
}
```

```
$(eval $(call exe_program,chord,.,chord.c,STACK=512,LIBS=snd))
```

13.3 Phrases

```
static snd_phrase_event_t tune[16];
uint16_t n;

snd_phrase_parse("| c4 d4 e4 ~ | g4@12 e4 c4 |", 6, tune, 16, &n);
```

A phrase is a short melody for one voice, written as text and parsed once, in the main loop, into an event array you own. The tokens are a pitch (`c d e f g a b`, an optional `#` or `b`, an octave 0-7), a pitch with a length (`g4@12`), a rest (`~` or `~@12`), and `|`, which is only a visual separator. The second argument is the default length in ticks. Anything else is refused. `snd_phrase_start` then compiles the events with an instrument into a voice program in a buffer you supply (`snd_phrase_compile` reports the size it needs), and the tick plays it exactly as it plays a note. The parser is its own member, so a program that only calls `snd_note_for` does not link it.

13.4 Playing a ZTR song

ZTR is the tracker's song format: 16 channels, orders, patterns, instrument macros and effects. Songs are written in the native tracker, or on the PC in CSM or Furnace and converted with `tools/snd/csmc.py` or `fur2ztr.py`. libsnd reads a song through a reader (§14.1), so the same code plays from a file, a stream, a KRES resource or memory. Error checks are left out here:

```
FILE *f = fopen("/fs/MUSIC/THEME.ZTR", "rb");
kos_reader_t song;

kos_reader_from_stdio(&song, f);
snd_open();
snd_ztr_open(&song);     /* reads and checks the song */
snd_ztr_prepare();       /* loads the first order */
snd_ztr_play();
/* timer events call snd_tick; the main loop calls snd_service */
snd_ztr_stop();
snd_ztr_close();         /* forgets the source; does not close it */
snd_close();
fclose(f);
```

- **Open** makes one forward pass over the file, keeps everything except the pattern data in memory and checks the song's CRC, so a damaged file fails before any register is written. A song too large for the library's tables, or with an order over 1,024 bytes of pattern data, is `-KOS_ENOSPC`. It is slow: more than half a second today, so open songs before the music must start.
- **Play** streams the patterns. `snd_service` reads the next order into the second of two 1,024-byte buffers while the current one plays, so the tick never reads the file.
- **Underrun.** If the next order is not ready when the song reaches it, the song mutes and enters `SND_ZTR_UNDERRUN`; the `snd_service` call that completes the order resumes it at its first row. A read error keeps the song in that state, never a false song end.
- **The source is borrowed.** Keep it open and unchanged until `snd_ztr_close`, then close it yourself.

`snd_ztr_pause` mutes and holds the song's state, `snd_ztr_resume` restores it, and `snd_ztr_stop` silences the song's voices and releases them. A song does not loop; play it again to repeat it. `snd_ztr_get_status` reports the state, order, row, tick rate, reserved channels, the last read error and the overrun and underrun counts.

13.5 Pre-rendered streams

A ZVG stream is one SN76489's register writes and waits, compiled on the PC from a VGM file. `snd_zvg_open(&reader, chip)` checks it and names the chip it

plays on (0-3); then `snd_zvg_prepare`, `snd_zvg_play`, `snd_zvg_stop` and `snd_zvg_close`, as for a ZTR song. `snd_service` keeps a 256-byte ring filled ahead of the decoder. A stall mutes the chip and playback resumes exactly where it stopped; a malformed or truncated stream stops with an error in `snd_zvg_get_status`. One ZVG, one ZTR song, or neither plays at a time.

13.6 Sharing voices

A playing ZTR song reserves every channel its orders use, and a ZVG stream its chip's four voices. Notes and phrases may use the voices left free. A note call on a reserved voice returns a busy result and leaves the song alone. Nothing is stolen or reallocated automatically.

13.7 Costs

ONE TICK	T-STATES AT 10 MHZ
12 voices waiting	about 207
12 voices each stepping their level	about 9,990
12 notes starting	about 18,600
a ZVG wait	about 230, plus about 27 per register byte written
a ZTR song, median	17,139 (Night Market); 11,133 (RunawayCircuit)
a ZTR song, worst	59,343 and 108,945: 36% and 65% of a 60 Hz frame

The note engine uses 768 bytes of RAM. RunawayCircuit drives all four PSGs and many simultaneous note starts on purpose; ordinary music sits nearer Night Market. Where a game cannot afford the ZTR tick, ship the music as ZVG.

PART II · 14

Readers and resource containers

Two SDK facilities for data: a **reader**, which lets a library read any source the same way, and **KRES**, a container that holds many named resources in one file. The card serves only two open files at once (chapter 16), and KRES is the way to keep many pieces of data behind one of them.

14.1 Readers

```
#include "kos/reader.h"
typedef struct kos_reader {
        void *context;
        kos_read_at_fn read_at;   /* count, 0 at EOF, or -error */
        uint32_t size;            /* exact length in bytes */
} kos_reader_t;
```

A reader is a borrowed source read at absolute offsets, with a known length. libsnd takes one wherever it reads a song. Build one with an adapter: `kos_reader_from_handle`, `kos_reader_from_stdio`, `kos_reader_from_kres` or `kos_reader_from_memory`. The handle and memory adapters take a small descriptor you own, `kos_handle_source_t` or `kos_memory_source_t`; keep it alive as long as the reader. No adapter closes its source and none adds a buffer: the stdio adapter uses the stream's own.

`kos_reader_read_full(reader, offset, dst, length)` repeats short reads until the range is complete, and returns the length, a negative error, or `-KOS_EIO` if the source ends first. Each adapter is its own member, so including the header links nothing; the KRES adapter lives in `libkres`.

14.2 KRES containers

A KRES container is one file holding up to 32 named resources. Each resource is a contiguous run of 512-byte records, and all of them are reached through the

container's one file handle. Names are 1-48 bytes, compared byte for byte; a `/` in a name is an ordinary byte, not a directory. The usual extension is `.KPR`.

There are two kinds. A **mutable** container is edited and committed by a program. A **sealed** one is built once and then only read: fonts, messages, game assets, a shipped project. Build with `LIBS=kres` and include `kos/kres.h` and `kos/kres_konaos.h`. Reading looks like this:

```
kres_install(&kres_io_konaos_read, &kres_cache_xmem, &ctx);   /* once */
if (kres_open(&game, "/fs/GAME.KPR", KRES_OPEN_READ) < 0)
        return 1;
if (kres_get(&game, "levels/1", &level, 0) == 0) {
        n = kres_read(&level, buffer, sizeof buffer);
        kres_rclose(&level);
}
kres_close(&game);
```

`game`, `level` and `ctx` are statics of the types the headers declare. Each open resource has its own cursor, moved with `kres_seek`; `kres_read_records` and `kres_write_records` move whole records by number without touching it. A process holds one container at a time. Its catalog is cached in a 2 KiB extended-memory object while it is open, so finding a resource costs no card read.

14.3 Changing a container

A program that writes installs the full adapter with `kres_install_mut(&kres_io_konaos, &kres_cache_xmem, &ctx)` and opens with `KRES_OPEN_WRITE`, adding `KRES_OPEN_CREATE` to make a new container.

- **Create and write.** `kres_create` allocates a resource with an initial capacity in records. A write inside the capacity changes it in place; one that outgrows it moves the resource to the end of the file. A write at the end of the data extends it, and a write past the end is refused: use `kres_resize`, which zero-fills, to make room.
- **Commit.** `kres_flush` and `kres_close` make every change durable. The catalog is written to its inactive copy before the header switches to it, so an interruption at any point leaves the last committed catalog intact. Check the result of `kres_close`.

- **One writer.** One resource is written at a time; others may be read meanwhile.
- **Compact.** Deleting or moving a resource leaves a hole that nothing reuses. `kres_compact(path, flags)` rebuilds the container in a temporary file and swaps the names; `KRES_TIGHT` also drops unused capacity and `KRES_SEAL` writes a sealed container. Every resource must be closed, the card needs room for the live data, and compaction holds both file slots while it runs.
- **Close it before `run()`.** The container keeps a card file open. Flush and close it before starting another program.

A sealed container is written with `kres_build_begin`, `kres_build_add`, `kres_build_write` and `kres_build_end`. An interrupted build leaves an invalid file, and `kres_build_abort` closes it. `kres_verify` checks a container and fails on anything malformed. `kres_extract` copies one resource into a new file, which is how a program stored in a container is run.

Errors are negated `KOS_E*` codes and four of the format's own: `KRES_E_FULL` (the catalog holds 32 already), `KRES_E_RECLIM` (the 65,535-record limit), `KRES_E_BAD` (not a KRES v1 file) and `KRES_E_GROW`.

14.4 What links

MEMBER	HOLDS
`kres_util`, `kres_read`, `kres_adapter`, `kres_cache_xmem`	install, the read path, the adapter's read side and the catalog cache: all a sealed-container reader links
`kres_mut`, `kres_adapter_mut`	mutation, flush and growth; the adapter's write, truncate, rename and unlink
`kres_build`	the sealed builder
`kres_compact`	rebuild compaction
`kres_extract`	one-resource export
`kres_verify`	the validation pass

On the PC, the KonaOS tree's host tool, `build/host/kres` (`make kres-host`), creates, lists, shows, imports, extracts, unpacks, packs, verifies, compacts and seals containers. It converts names to and from CP850 at the boundary and copies payloads byte for byte.

PART III

Running

Starting programs from the shell and the monitor, what the monitor can show you, the limits you will meet, and the example programs read end to end.

PART III · 15

Running and debugging

15.1 From the shell and the monitor

The shell, `/system/shell.exe`, is the production boot program. Type a program's name and arguments; it searches `.`, `/fs/BIN` and `/system`, with or without `.exe`, and reports the child's status when it exits. The shell shows `/fs` as `/` and `//` names the real root; in code, always use the real path (§2.4). Type `monitor` to start the monitor as a child.

Its built-ins are `cd pwd echo path help exit reboot` and the file commands `ls cat stat cp mv rm mkdir rmdir`. The line editor keeps the last five lines on the up and down arrows for the session, and CTRL-L clears the screen and redraws the prompt and line. On every start the shell prints its banner and logo, so seeing the banner again means your program exited and the shell was relaunched.

At the monitor's `Z80>` prompt:

```
Z80> X /fs/BIN/HELLO.EXE one "two three"
Hello, KonaOS!
...
exec: exit status 0x0000
```

`X path [tail]` runs a program from `/system` or `/fs` through the ordinary exec path and passes the tail byte-exact. A program that cannot start prints `ERR: <code>`.

15.2 Acceptance and production images

An image's system file names its boot program and what follows a normal exit:

IMAGE	AFTER_EXIT	AFTER THE BOOT PROGRAM EXITS
production	`relaunch`	`boot: /system/shell.exe exit 0`, then the shell starts again
acceptance (for example the demo image)	`monitor`	`boot: /system/hello.exe exit 0`, then the monitor

Either way, a boot program that cannot start prints `boot: <path> failed <n>` and falls back to the monitor. Every start of the boot program begins with a console reset, so your program never inherits a terminal another program left behind. An acceptance image is the convenient way to test a program that should run once at boot and leave you at the monitor.

15.3 What the monitor shows without kernel knowledge

COMMAND	USE IT TO
`R`	see ABI version, common base, the monitor's own pid and bank, and the interrupt report. If an event handler never runs, read the `irq:` line: `last 02` means the clock is ticking
`DB bank addr len`	dump any bank 0-6, `0000h-DFFFh`
`D addr len`, `M addr value`	dump or set the monitor's own bank
`I port`, `O port value`	read or write I/O ports `10h-FFh`
`X path [tail]`	run a program and see its status
`B`	warm reboot

A post-mortem technique: have your program print its bank from `getinfo`. Banks are not cleared when a program exits, so after it returns you can inspect its statics with `DB`, provided no other program has loaded into that bank since. Your map file gives the addresses.

15.4 Finding a silent crash

A program that faults loses output it has not flushed, so a crash can look as if nothing ran. Bracket the fault with markers, flushing after each:

```
printf("@1\r\n"); console_flush();
load_level();
printf("@2\r\n"); console_flush();
draw_level();
printf("@3\r\n"); console_flush();
```

The last marker printed brackets the fault. Move the markers into the bracketed code and repeat until the failing statement is isolated.

15.5 Taking the console over the serial line

On the production console, three ESC bytes sent from a terminal on the serial line move keyboard input to the serial line and start mirroring console output to it. Three more ESCs give input back to the keyboard; the mirror stays on until the next reboot. Use it to drive a program, or watch its output, from a host terminal.

- The gesture is noticed only when the running program calls a console input function, so a program that never reads input does not see it.
- While the mirror is on, the console refuses render transactions, so a TUI program falls back to ANSI output and draws on both screens (§11.5).
- Only the third ESC of a run is consumed; the first two arrive as data, so arrow and function keys from the terminal still work.

PART III · 16

Limits you will hit

LIMIT	VALUE	WHEN EXCEEDED
path length	127 bytes	`-KOS_EINVAL`, refused before the call
command tail	125 bytes	`-KOS_EINVAL` from `run`
`argv` words	16	extra words are not stored
`console_readline` line	80 characters	a bell
card file names	8.3, upper-cased	refused
open files on `/fs`	2 files, 1 directory	`-KOS_EMFILE`
handle table	12 entries, shared with children	`-KOS_EMFILE`
streams	4 (but only 2 card files at once)	`fopen` returns `NULL`
one IPC buffer	4,096 bytes	`-KOS_EINVAL`
event subscriptions	4	`-KOS_ENOSPC`
`run` nesting	4 user processes	`-KOS_EBUSY`
stack in a far program	8 KiB	refused at pack time
far-code or mappable segment	16 KiB each	refused at pack time
far segments per program	5 banks, fewer if a parent holds banks	`-KOS_ENOSPC` at run time
`xalloc`	32 KiB; 16 KiB mappable	`-KOS_ENOSPC` / `-KOS_EINVAL`
segments per EXE	8 rows, 20 slots	refused at pack time
TUI window stack	4	`-KOS_EINVAL`
TUI capacity	85 × 26 or 128 × 35	build error
TUI dialog	60 × 16, one at a time	a nested call returns `TUI_ID_NONE`
graphics command queue	32 commands	flushed automatically, in order
`libsnd` catch-up	16 ticks per call	the excess is counted as an overrun, not replayed
ZTR song	65,535 bytes; 1,024 pattern bytes per order	`-KOS_ENOSPC` from `snd_ztr_open`
KRES container	32 resources, names of 1-48 bytes	`KRES_E_FULL`
open KRES containers	1 per process	refused

LIMIT	VALUE	WHEN EXCEEDED
exclusive video hold	1	`-KOS_EBUSY`
exclusive sound hold	1	`-KOS_EBUSY`

16.1 Errors you actually see

ERROR	USUAL CAUSE
`-KOS_ENOENT`	wrong path; remember card names are 8.3
`-KOS_ENODEV`	no card behind `/fs`, or the driver is not in this build
`-KOS_EINVAL`	path too long or empty, a buffer outside `0100h-DFFFh`, a bad handle, or no standard handles
`-KOS_EMFILE`	a third card file open, or the handle table full
`-KOS_EROFS`	writing under `/system`
`-KOS_EXDEV`	`rename` between `/fs` and `/system`
`-KOS_ENOSPC`	no bank for a child or its far segments; a full table
`-KOS_EBUSY`	`run` nested too deeply; a device held exclusively
`-KOS_ENOTSUP`	subscribing on a device that serves no events; a console without render transactions
`-KOS_EFAULT`	the program faulted or returned without exiting
`KOS_RT_EFARMAP` (`30h`)	a banked call to a function that is not in a far segment

PART III · 17

Examples walk-through

`hello`, `fardemo`, `fardata` and `files` ship in the SDK's `examples/` directory, with the three TUI examples. §1.5 and §1.6 are written for this guide.

17.1 `hello`

Prints a greeting and its arguments. It links exactly `crt0`, the runtime, `args`, the console client and its gates, `printf` with its formatters and the division helpers, and no filesystem, exec or diagnostic member. It is the baseline for what an empty program costs.

17.2 `fardemo`

Walked through in §7.1.

17.3 `fardata`

`fardata.c`, with `levels.c` as `DATA1` and `fardata_far.c` as `FAR1`. It maps the data segment, reads a 4 KiB table and writes a zeroed 2 KiB array, unmaps, and only then makes a banked call with values copied out of the table. It is the one-window rule in one program.

17.4 `files`

Opens, reads and lists files. The listing loop has this shape:

```
struct kos_ns_entry e[4];
unsigned int cookie = 0;
int n, i;

while ((n = list("/fs", &cookie, e, 4)) > 0)
        for (i = 0; i < n; i++)
                printf("%s\r\n", e[i].name);
```

17.5 A stream and the timer

Counts the lines in a file and times the work. It shows a buffered stream, an explicit `kos/timer.h` include, unsigned elapsed time, and turning a negative error into a positive exit status.

```
#include "kos.h"
#include "kos/timer.h"

int main(int argc, char **argv)
{
        FILE *f;
        int timer;
        uint32_t t0, t1;
        char line[81];
        unsigned int n = 0;

        if (argc < 1) {
                printf("usage: lines path\r\n");
                return 1;
        }
        timer = timer_open();
        if (timer < 0)
                return -timer;
        f = fopen(argv[0], "r");
        if (f == NULL) {
                printf("cannot open %s\r\n", argv[0]);
                return 2;
        }
        timer_now(timer, &t0);
        while (fgets(line, sizeof line, f) != NULL)
                n++;
        timer_now(timer, &t1);
        fclose(f);
        printf("%u lines in %lu us\r\n", n, timer_us(t1 - t0));
        return 0;
}
```

```
$(eval $(call exe_program,lines,src,lines.c))
```

The timer handle is not unbound explicitly: exit releases every handle.

17.6 A menu with a confirmation

The SDK ships three TUI programs: `tui_demo` (panels, a nested window, styles, frames, a status row and a key loop), `tui_keys` (prints the name of every decoded key, which is the quickest check of a terminal) and `tui_dialog` (menu, list, input, checkbox, buttons and a modal confirmation). This shorter program shows the widget pattern they share:

```
#include "kos.h"
#include "tui_widgets.h"

#define ID_OPEN 10
#define ID_SAVE 11
#define ID_QUIT 12

static const struct tui_menu_item items[] = {
        { "Open", 'o', ID_OPEN },
        { "Save", 's', ID_SAVE },
        { "Quit", 'q', ID_QUIT },
};

int main(void)
{
        tui_id_t id;

        if (tui_init() < 0)
                return 1;
        for (;;) {
                id = tui_menu_run(items, 3);
                if (id =/= ID_QUIT && id =/= TUI_ID_NONE)
                        continue;       /* open and save left out */
                if (tui_confirm("Quit", "Leave the program?") == TUI_ID_YES)
                        break;
        }
        tui_shutdown();
        return 0;
}
```

```
$(eval $(call exe_program,menu,.,menu.c,STACK=512,LIBS=tui_widgets:tui))
```

The menu table is `const`, so it is not initialised writable data. The confirmation restores the screen it covered before it returns.

APPENDIX A

Quick reference of the SDK headers

HEADER	VIA KOS.H	PROVIDES
kos/console.h	yes	exit reboot getinfo putchar getchar console_write console_read console_poll console_flush console_puts console_readline console_info console_render
kos/fs.h	yes	open close read write seek sync truncate stat mkdir rmdir unlink rename; KFS_O_*, KFS_SEEK_*, struct kfs_stat
kos/ipc.h	yes	bind call unbind list chdir getcwd; struct kos_ipc_request
kos/process.h	yes	startup splitargs run idle idle_input idle_input_ready; KOS_ARGV_MAX
kos/seg.h	yes	seginfo segmap farcall farmap farunmap blob xload; KOS_RT_EFARMAP
kos/xmem.h	yes	xalloc xfree xread xwrite xinfo xmap aperture_open aperture_close
klibc/string.h	yes	strlen strcmp strncmp strchr strrchr strlcpy strlcat memcpy memmove memset memcmp
klibc/fmt.h	yes	printf snprintf utoa ultoa itoa ltoa
klibc/stdio.h	yes	FILE fopen fclose fread fwrite fgetc getc fputc putc fgets fputs fflush feof ferror fseek ftell rewind
kos/timer.h	no	timer_open timer_now timer_us timer_wait_us
kos/event.h	no	notify timer_event; kos_event_handler_t
kos/stream.h	no	fmeasure fstats stream_plan; struct kos_stream_need, kos_stream_plan
kos/sound.h	no	sound_open sound_close sound_info sound_write sound_reset sound_out; struct ksound_write, ksound_info
tui.h	no; LIBS=tui	lifecycle, cursor, text, windows, styles, frames, keys, save and restore (chapter 11)
tui_widgets.h	no; LIBS=tui_widgets:tui	status bar, menu bar, label, button, input, checkbox, list box, menus and dialogs
kos/gfx.h	no; LIBS=gfx	modes, pages, drawing, batching, upload, palette, text and glyphs (chapter 12)
kos/kimg.h	no	the KIMG v1 picture header
kos/snd.h	no; LIBS=snd	notes, instruments, phrases, ZTR and ZVG playback (chapter 13)
kos/reader.h	no	kos_reader_t, kos_reader_read_full and the four adapters

HEADER	VIA KOS.H	PROVIDES
kos/kres.h, kos/kres_konaos.h	no; LIBS=kres	KRES containers and the KonaOS adapter and cache (chapter 14)
kos/diag.h	no	diag_mem_read diag_mem_write diag_io_in diag_io_out diag_bank_info diag_system_info; recovery tools only

APPENDIX B

Error codes

SDK wrappers return these negated. One numbering covers the whole system (abi/kos/errno.h).

VALUE	NAME	MEANING
01h	KOS_ENOSYS	no such syscall
02h	KOS_EINVAL	invalid argument, handle or descriptor
03h	KOS_ENODEV	no device, or no media
04h	KOS_EBUSY	busy, owned, or a nesting limit
05h	KOS_ENOENT	not found
06h	KOS_ENOSPC	out of space, memory or table entries
07h	KOS_ENOTSUP	not supported by this object
08h	KOS_ETIMEDOUT	a bounded wait expired
09h	KOS_EAGAIN	nothing available yet
0Ah	KOS_EIO	corrupt, truncated or failed I/O
0Bh	KOS_EINIT	initialisation refused
0Ch	KOS_EFAULT	a process fault
10h	KOS_EEXIST	already exists
11h	KOS_ENOTDIR	not a directory
12h	KOS_EISDIR	is a directory
13h	KOS_ENOTEMPTY	directory not empty
14h	KOS_EROFS	read-only
15h	KOS_ESTALE	stale object identity
16h	KOS_EXDEV	across providers
17h	KOS_EMFILE	handle table or open-file limit reached
30h	KOS_RT_EFARMAP	SDK-level: a banked call that names no far segment

APPENDIX C

EXE geometry rules that affect link success

The packer applies every check the loader applies, so a program that would be refused at run time is refused at build time instead. These are the ones a program author can trip.

Image and BSS end at or below `E000h`

`0100h + image_size ⩽ E000h` and `image_end + bss_size ⩽ E000h`. Large static buffers count against BSS.

The stack does not collide with BSS

The stack base, `E000h - STACK`, must be at or above the end of BSS. Raising `STACK` can break a link that worked.

No initialised writable data

`_INITIALIZER` and `_INITIALIZED` must be empty. Make the object `const` or set it at run time. In a `DATAn` source the packer names the object.

The far link rule

With hand-declared `FARn` or `DATAn` segments: image and BSS below `8000h`, stack base at or above `C000h`, each far area based at `8000h` and at most 16 KiB.

The placed link rule

In a placed program a reference is direct only when its target is resident or in the referrer's own zone; other calls go through veneers, and data or code addresses cross zones only to resident targets. The packer names the module and symbol that broke the rule. An event handler belongs in `RESIDENT=`; a function whose address is taken must be resident.

Segment table bounds

At most 8 segments and 20 slots of 8 KiB. `DATA` rows up to 32 KiB, far-code and mappable rows up to 16 KiB.

The scratch stays in the argument window

`kos_scratch` must lie in `0100h-1FFFh`. The SDK pins it at the start of its low-data area, so this fails only if you move `LOWBASE`.

Refused at run time, not at build

A wrong ABI version (`-KOS_EINVAL`) and too few free banks for the far segments (`-KOS_ENOSPC`).